Università di Genova

Embedded Systems

Protocols

Prof. Enrico Simetti

Transferring information: an example

Consider this scenario: A microcontroller manages a temperature, humidity and pressure sensors.

Periodically, this information must be sent to a computer to be displayed to the user.

How do we transfer this information?

Transferring information: an example

Microcontroller side:

```
int temperature = 25;
int humidity = 80;
int pressure = 1000;

write_uart(temperature);
write_uart(humidity);
write_uart(pressure);
```

PC side:

```
int temperature;
int humidity;
int pressure;

read_uart(&temperature);
read_uart(&humidity);
read_uart(&pressure);
```

Transferring information: an example

Microcontroller side:

Address	Memory cell	Variable name	Value
0x1000	0x19	*temperature*	25
0x1001	0x00		
0x1002	0x50	*humidity*	80
0x1003	0x00		
0x1004	0xE8	*pressure*	1000
0x1005	0x03		

Transferring information: an example

This is the stream of bytes going out

byte n#	0	1	2	3	4	5
value	0x19	0x00	0x50	0x00	0xE8	0x03
	temperature		*humidity*		*pressure*	

What happens if the first byte is lost for some reason?

Transferring information: an example

PC side:

Address	Memory cell	Variable name	Value
0xF000	0x00	*temperature*	20480
0xF001	0x50		
0xF002	0x00	*humidity*	-6144
0xF003	0xE8		
0xF004	0x03	*pressure*	6403
0xF005	0x19		

Disaster... data is corrupted.

It is not as simple as it seems!

Protocol Types

Two kind of protocols:

- binary protocols: the information is sent as bytes (like in the previous example);
  - pro: it takes the least number of bytes to send the information;
  - con: cannot be read by humans easily;
  - con: compatibility between architectures must be considered (e.g. byte endianess);
- ascii protocols: the information is sent as characters;
  - pro: can be read by humans easily;
  - pro: less problems for compatibility between different architectures (since the char is always one byte);
  - con: it takes more bytes to send the same information
  - con: requires string parsing

Characters vs Bytes

But aren't characters bytes as well?

Address	Memory cell	Variable name	Value
0x1000	0x54	*msg[0]*	'T'
0x1001	0x3D	*msg[1]*	'='
0x1002	0x32	*msg[2]*	'2'
0x1003	0x35	*msg[3]*	'5'
0x1004	0x2C	*msg[4]*	','
0x1005	0x48	*msg[5]*	'H'
0x1006	0x3D	*msg[6]*	'='
0x1007	0x38	*msg[7]*	'8'
0x1008	0x30	*msg[8]*	'0'

ASCII Stream

This is the stream of bytes going out

byte n#	0	1	2	3	4	5	6	7	8
value	0x54	0x3D	0x32	0x35	0x2C	0x48	0x3D	0x38	0x30
	T	=	*2*	*5*	,	*H*	=	8	0

To send temperature (2 bytes) and humidity (2 bytes) we need 8 bytes!

NMEA Protocol

ASCII protocol

format: $PREFIX,data1,data2,...,dataN*CHECKSUM

example: $GPRMC,225446,A,4916.45,N,12311.12,W,000.5,054.7,191194,020.3,E*68

225446 -> Time of fix 22:54:46 UTC

A -> Navigation receiver warning A = OK, V = warning

4916.45,N -> Latitude 49 deg. 16.45 min North

12311.12,W -> Longitude 123 deg. 11.12 min West

000.5 -> Speed over ground, Knots

054.7 -> Course Made Good, True

191194 -> Date of fix 19 November 1994

020.3,E -> Magnetic variation 20.3 deg East

*68 -> mandatory checksum

Parser Implementation

- **format: $MCSEN,temp,hum,press***
- **bytes are received one at a time**
  - discard bytes until a $ is received
  - read chars and put them in a string called msg_type until a comma is read
  - read chars and put them in a string called msg_payload until an asterix is read
  - when an asterix is read, return a value to identify that a new message has been received
  - reset state and go back to point 1)
- **the role of this parser is to extract the message type and the payload from the "frame", i.e. removing the $ and * which are used to delimit the message during the serial transmission**

HDLC Binary protocol

High-Level Data Link Control (HDLC) protocol

- delimiter start of packet 1 byte
- address 2 bytes
- control 2 bytes
- payload n bytes
- checksum 2/4 bytes
- delimiter end of packet 1 byte

delimiters must appear only at the beginning and end of the frame

Example of Binary Protocol

- Start of Frame 0x7E
- Payload n bytes
  - ID packet 2 byte
  - payload n-2 bytes
- End of Frame 0x7F

Binary Transmission Example

Suppose we want to transmit packet ID = 2, and the payload are two integers of two bytes: 100 and 30000:

byte n#	0	1	2	3	4	5
value	0x02	0x00	0x64	0x00	0x30	0x75
	2		*100*		*30000*	

Binary Transmission Example

The "frame" is thus:

byte n#	SOF	0	1	2	3	4	5	EOF
value	**0x7E**	0x02	0x00	0x64	0x00	0x30	0x75	**0x7F**
		2		*100*		*30000*		

Binary Transmission Example

Open problems:

- What if 0x7E or 0x7F are found inside the payload?
- Consider now packet ID = 2, and the payload are two integers of two bytes: 126 and 32016:

byte n#	0	1	2	3	4	5
value	0x02	0x00	0x7E	0x00	0x10	0x7D
	2		*126*		*32016*	

Byte Stuffing

Byte stuffing or escaping:

- whenever we find an unallowed byte inside the payload we 'stuff' (add) one byte (0x7D) before that one
- we also modify the byte by doing a XOR with a mask (0x80)

Byte Stuffing Example

We escape 0x7E, adding 0x7D in front and doing the 0x7E XOR 0x80 = 0xFE

byte n#	0	1	s	2	3	4	5
value	0x02	0x00	0x7D	0xFE	0x00	0x10	0x7D
	2			*126*		*32016*	

Anyone notice a problem?

Byte Stuffing Example

The 0x7D byte must also be escaped:

byte n#	0	1	s	2	3	4	s	5
value	0x02	0x00	0x7D	0xFE	0x00	0x10	0x7D	0xFD
	2			*126*		*32016*		

Byte Stuffing Example

Finally, we include the start and end of frame delimiters:

byte n#	**SOF**	0	1	s	2	3	4	s	5	**EOF**
value	**0x7E**	0x02	0x00	0x7D	0xFE	0x00	0x10	0x7D	0xFD	**0x7F**
		2			*126*		*32016*			

the receiver will need to do the inverse operation!

Binary Parser Implementation

- format: 0x7E payload 0x7F
- bytes are received one at a time
  - discard bytes until a 0x7E is received
  - read bytes and put them in an array called payload until a 0x7F is read
  - when 0x7F is read, unescape the bytes received and return a value to identify that a new message has been received
  - reset state and go back to point 1)
- the role of this parser is to extract the message type and the payload from the "frame", i.e. removing the start and end of frame delimiters, as well as any escape byte.

Receiving Binary Data

Open problems:

- now we have an array of chars (bytes)
- how do we extract the values?

Receiving Binary Data

```
int n = 10;
char bytes[2] = {0x30, 0x75};
char* pn = (char*) &n;
memcpy(pn, bytes, 2); // memcpy(destination, source,
size)
```

Before the memcpy:

Address	Memory cell	Variable name	Value
0x1000	0x0A	*n*	10
0x1001	0x00		
0x1002	0x30	*bytes[0]*	48
0x1003	0x75	*bytes[1]*	117
0x1004	0x00	*pn*	0x1000
0x1005	0x10		

Receiving Binary Data

```
int n = 10;
char bytes[2] = {0x30, 0x75};
char* pn = (char*) &n;
memcpy(pn, bytes, 2); // memcpy(destination, source,
size)
```

After the memcpy:

Address	Memory cell	Variable name	Value
0x1000	0x30	*n*	30000
0x1001	0x75		
0x1002	0x30	*bytes[0]*	48
0x1003	0x75	*bytes[1]*	117
0x1004	0x00	*pn*	0x1000
0x1005	0x10		



Università
di Genova